

Laboratório de Concorrência · Aula 10

André Filipe Caetano Mulati · RA 24011149-2 · Turma B

Gerado em 09/10/2026, 20:24:27 · tempo registrado no cronômetro: 35:23

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

Cooperação: acontece quando pessoas trabalham juntas para alcançar um objetivo em comum. Exemplo: pessoas carregando um móvel juntas.

Competição: acontece quando pessoas disputam algo, buscando alcançar seus próprios objetivos. Exemplo: dois atletas disputando uma corrida.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

2.000.000

1ª execução (n = 1.000.000)

1.543.276

2ª execução (n = 1.000.000)

1.672.891

3ª execução (n = 1.000.000)

1.389.450

Saída do programa

esperado: 2000000 obtido: 1543276

esperado: 2000000 obtido: 1672891

esperado: 2000000 obtido: 1389450

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Não. Os valores podem ser menores que o esperado e variar entre as execuções porque as duas threads acessam e alteram o mesmo contador ao mesmo tempo.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

A thread 1 lê o valor 0 e a thread 2 também lê 0. As duas somam 1, obtendo 1. Quando gravam o resultado, uma sobrescreve a outra. Assim, uma soma se perde.

1.3 Com n = 1000, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Com $n = 1000$, o resultado pode ser igual ou próximo do esperado, pois há menos operações. Esse erro é perigoso porque pode não aparecer nos testes, mas acontecer em produção, causando resultados incorretos.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

AAABBB $\rightarrow$ 8

Saída do programa

AAABBB $\rightarrow$ 8

BBBAAA $\rightarrow$ 7

ABABAB $\rightarrow$ 4

Ordens possíveis: 20

TOTAL = 4: 6 ordens, por exemplo AAB BBA

TOTAL = 6: 5 ordens, por exemplo AABABB

TOTAL = 7: 4 ordens, por exemplo ABBBAA

TOTAL = 8: 5 ordens, por exemplo AAABBB

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

São 20 ordens possíveis, com 4 resultados diferentes (4, 6, 7 e 8). 5 ordens produzem o resultado correto (8).

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Na ordem AAB BBA:

A lê 3 e calcula 4.

A grava 4.

B lê 4 e calcula 8.

A lê 4 e calcula 5.

B grava 8.

A grava 5.

O resultado final é 5, não 4. Para obter 4, podemos usar ABABAB: as operações se sobrepõem, fazendo com que atualizações sejam perdidas.

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Acredito que sem sincronização será a mais rápida, mas poderá apresentar um resultado incorreto. O `synchronized` e o `AtomicInteger` devem chegar ao resultado correto, porém podem demorar mais por causa da sincronização.

Saída do programa

esperado: 2000000 sem sincronizacao: 1456789 em 15 ms synchronized: 2000000 em 180 ms
AtomicInteger: 2000000 em 95 ms

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

As versões synchronized e AtomicInteger deram o valor correto (2.000.000). Elas costumam ser mais lentas porque precisam controlar o acesso ao contador para evitar erros.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não. Um AtomicInteger em cada saldo não garante que a transferência inteira aconteça corretamente. É preciso sincronizar as duas operações juntas, retirando de uma conta e depositando na outra, usando, por exemplo, synchronized ou um bloqueio (Lock).

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

Não. As mensagens "A terminou" e "B terminou" provavelmente não aparecerão, porque as duas threads ficam esperando recursos que estão presos uma à outra, causando um deadlock (impasse).

Saídas (antes e depois da correção)

antes:
IMPASSE: 2 threads presas

depois:
A terminou B terminou sem impasse

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

A thread A pegou o recurso X e esperava pelo Y. A thread B pegou o recurso Y e esperava pelo X. Como cada uma segurava o recurso que a outra precisava, as duas ficaram presas.

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Pegar os recursos sempre na mesma ordem evita que uma thread fique esperando recursos na ordem inversa da outra, eliminando esse tipo de impasse. Um exemplo real é um sistema bancário em que duas transferências tentam acessar as mesmas contas ao mesmo tempo.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

3 trabalhadores: cerca de 200 ms.

1 trabalhador: cerca de 600 ms.

6 trabalhadores: cerca de 100 ms.

Tempo com 1 trabalhador

600 ms

Tempo com 3 trabalhadores

200 ms

Tempo com 6 trabalhadores

100 ms

Saída do programa

```
trabalhador 1 fez a tarefa 1 trabalhador 2 fez a tarefa 2 trabalhador 3 fez a tarefa 3  
trabalhador 1 fez a tarefa 4 trabalhador 2 fez a tarefa 5 trabalhador 3 fez a tarefa 6 3  
trabalhadores, 6 tarefas de 100 ms: levou 200ms
```

5.1 Explique os três tempos. Por que este programa não precisa de `synchronized` nem de trava?

1 trabalhador: leva cerca de 600 ms, pois executa as 6 tarefas uma após a outra.

3 trabalhadores: leva cerca de 200 ms, pois cada trabalhador executa 2 tarefas.

6 trabalhadores: leva cerca de 100 ms, pois cada trabalhador executa uma tarefa ao mesmo tempo.

O programa não precisa de `synchronized` nem de trava porque as tarefas são distribuídas por um canal, evitando o compartilhamento direto de dados entre os trabalhadores.

Saídas (antes e depois da troca)

```
antes: fatal error: all goroutines are asleep - deadlock!
```

```
depois: 1
```

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

O primeiro programa trava porque o canal não tem espaço para armazenar a mensagem, então o envio fica esperando alguém receber.

Com `make(chan int, 1)`, o canal consegue armazenar uma mensagem. Assim, o envio acontece sem travar e o programa imprime 1.

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

Acredito que o programa não vai compilar, porque o Rust não permite que duas threads alterem ou compartilhem uma variável sem a proteção adequada. Para resolver, é preciso usar mecanismos como `Arc` e `Mutex`.

Saídas (o erro e o resultado com `move`)

```
sem move: error[E0373]: closure may outlive the current function, but it borrows contador,  
which is owned by the current function help: to force the closure to take ownership of  
contador (and any other referenced variables), use the move keyword
```

```
com move: 0
```

Saída do programa

```
contador = 400000
```

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

Porque move transfere o valor do contador para a thread, que altera sua própria cópia. O contador original continua valendo 0.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java, o erro de concorrência aparece durante a execução. Em Rust, o compilador detecta problemas de compartilhamento antes de executar o programa.

Vantagem: Rust evita muitos erros antes da execução.

Custo: exige mais cuidado e código adicional para compartilhar dados entre threads.

Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

A, C, D, B

Saída do programa

```
A
D
C
B
E: timer de 10 ms rodou depois de aproximadamente 300 ms
```

7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

A ordem foi A, D, C, B. B saiu depois porque o setTimeout executa quando o código atual termina e as promessas pendentes (C) são processadas primeiro.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

O timer de 10 ms atrasou porque o cálculo pesado bloqueou a única thread por cerca de 300 ms. Em um servidor Node.js, isso pode atrasar as respostas para muitos usuários ao mesmo tempo.

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

Pode ocorrer uma condição de corrida, vendendo o mesmo ingresso para duas pessoas. Para evitar isso, usaria sincronização para garantir que apenas uma compra seja processada por vez.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi como duas tarefas simultâneas podem causar erros mesmo em operações simples.